Instructions for Submission

1. File Naming and Submission:
   a. Create a **ZIP file** containing all your solution files.
   b. Name the ZIP file as `lab10_<ERP>.zip`, where `<ERP>` is your ERP.
   c. Ensure that the ZIP file contains:
      i. C++ source files
      ii. No unnecessary files (such as executables, temporary files, or editor configs)
   d. Do not include `TEXT(.txt)` files.
2. Code Neatness:
   a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
   b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
   c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
   a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
   b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
   a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
   b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
   c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
   a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
   b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
   a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Pointers in C++

Example 1: Basic Pointer Usage

```cpp
#include <iostream>

int main() {
    int a = 1; // Somewhere in memory, a block is allocated for an integer variable 'a',
        initialized to 1.
    int *ptr;  // A block of memory is allocated for a pointer to an integer (int *).

    /*
        Let's make the pointer 'ptr' store the address of the variable 'a'.
        This is achieved using the address-of operator (&).
    */
    ptr = &a; // 'ptr' is assigned the memory address of 'a'.

    // Print the address of 'a' using '&a'.
    std::cout << "Address of a: " << &a << std::endl;

    // Print the value of 'ptr', which is the address of 'a'.
    std::cout << "Value of ptr (address of a): " << ptr << std::endl;

    /*
        Now, dereference 'ptr' using the '*' operator to access the value stored at the memory
            location 'ptr' points to.
    */
    std::cout << "Value at address ptr (value of a): " << *ptr << std::endl;

    /*
        To get the address of the pointer variable 'ptr' itself:
        Recall that 'ptr' is also a variable stored in memory!
    */
    std::cout << "Address of ptr: " << &ptr << std::endl;

    return 0;
}
```

What Happens in Memory?

```
        +----------------+
        |                | 0x7
        +----------------+
        |                | 0x6
        +----------------+
        |                | 0x5
        +----------------+
        |                | 0x4
        +----------------+
        |                | 0x3
        +----------------+
        |   ptr = 0x1    | 0x2
        +----------------+
        |     a = 1      | 0x1
        +----------------+

Address of 'a' = 0x1
Value of 'a' = 1

Address of 'ptr' = 0x2
Value of 'ptr' = 0x1
```

Example 2: Modifying Values Through Pointers

```cpp
#include <iostream>

int main() {
    int a = 10; // Initialize an integer variable 'a' with the value 10.
    int *ptr = &a; // Pointer 'ptr' stores the address of 'a'.

    // Print the initial value of 'a'.
    std::cout << "Initial value of a: " << a << std::endl;

    // Modify the value of 'a' through the pointer.
    *ptr = 20; // Dereference 'ptr' and assign a new value to 'a'.

    // Print the updated value of 'a'.
    std::cout << "Updated value of a: " << a << std::endl;

    return 0;
}
```

```
Initial value of a: 10
Updated value of a: 20
```

Explanation:

- `int *ptr = &a;` makes `ptr` point to the memory address of `a`.
- `*ptr = 20;` modifies the value stored at the memory address `ptr` points to, which is the value of `a`.

Example 3: Pointer to a Pointer

You can have a pointer that points to another pointer.

```cpp
#include <iostream>

int main() {
    int a = 5;         // An integer variable 'a'.
    int *ptr = &a;     // 'ptr' points to the address of 'a'.
    int **pptr = &ptr; // 'pptr' points to the address of 'ptr'.

    // Print the value of 'a' through different levels of pointers.
    std::cout << "Value of a: " << a << std::endl;
    std::cout << "Value of a using ptr: " << *ptr << std::endl;
    std::cout << "Value of a using pptr: " << **pptr << std::endl;

    return 0;
}
```

```
Value of a: 5
Value of a using ptr: 5
Value of a using pptr: 5
```

Example 4: Passing Arrays Using Pointers

```cpp
#include <iostream>

// Function that prints all elements of an integer array
void printArray(int *arr, int size) {
    std::cout << "Array elements: ";
    for (int i = 0; i < size; ++i) {
        // Access elements using pointer arithmetic
        std::cout << *(arr + i) << " ";    // Equivalent to arr[i]
        // std::cout << arr[i] << " ";     // This works the same way
    }
    std::cout << std::endl;
}

int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // Pass the array to the function (decays to pointer)
    printArray(numbers, size);

    return 0;
}
```

```
Array elements: 10 20 30 40 50
```

Explanation:

- When an array is passed to a function, it decays to a pointer to its first element.
- `int *arr` in the function receives the address of `numbers[0]`.
- Using pointer arithmetic (`*(arr + i)`) or array indexing (`arr[i]`), we can access each element.
- Array indexing `arr[i]` is just syntactic sugar for `*(arr + i)`.
- This allows efficient manipulation of array elements without copying the entire array.

Wrapping Up

- The `*` operator is used for defining pointers and accessing the value at the memory address a pointer holds (dereferencing).
- The `&` operator retrieves the memory address of a variable.
- When arrays are passed to functions, they decay to pointers, and elements can be accessed either using pointer arithmetic (`*(arr + i)`) or array indexing (`arr[i]`).
- Pointers provide a way to reference and manipulate memory directly, which underlies many fundamental C++ operations such as function arguments and multi-level pointers.

Lab Tasks

Note: For Tasks 1-9, write clear line-by-line comments explaining the process and memory layout of variables.

Task 1: Array of Integers and Pointers

Define an array of integers and an array of pointers of the same size. Assign each pointer to point to the corresponding element in the integer array. Print all elements of the integer array using the array of pointers.

Task 2: One pointer to rule them all...

Define a fixed-size array of any data type (integer, float, or char). Declare a pointer to the first element of the array. Use the pointer to traverse the array and print each element.

Task 3: Swapping Values Using Pointers

Declare two variables of the same data type. Swap their values using pointers. Print the values before and after swapping.

Task 4: Initializing a Variable via Pointer

Write a function that takes a pointer to an integer and sets its value to `123`. In `main()`, declare an integer initialized to `0`, call the function, and print the updated value.

Task 5: There's nothing THERE!

Write a function that takes a pointer to an integer. Inside the function, declare a local integer (`int i = 2;`) and assign its address to the pointer. In `main()`, attempt to access the value through the pointer. Add comments explaining what happens and why this is problematic.

Task 6: Observing Pointer Dereferencing Behavior

Experiment with the following:

1. Declare `int i = 2` and attempt to dereference it. Explain the outcome.
2. Declare an array `int arr[]` and attempt to dereference it using `&arr`. Explain what happens.
3. Illustrate the memory layout for both cases and comment on why the behavior differs.

Task 7: Memory Addresses in Functions and Recursion

Write two functions `foo(int a)` and `bar(int& a)` that take an integer as input and print the memory address of `a`. Then write recursive versions of these functions: `fooRec(int a)` and `barRec(int& a)` that recursively call themselves `a` times. In each call print the value and address of `a`.

In `main()`, declare an integer variable and print its address. Call all four functions with this variable. Observe and comment on the differences in memory addresses when passing by value versus passing by reference, and how recursion affects the addresses.

Task 8: Sum of an Array via Pointer

Write a function that takes a pointer to an array of integers and its size, calculates the sum of all elements, and returns it. In `main()`, define an array of integers, call this function with the array, and print the resulting sum.

Task 9: Return Address Conditionally

Write a function `isGreater(int& a, int& b)` that takes two integers by reference. The function should return the **memory address** of `a` if `a > b`, and `nullptr` otherwise.

In `main()`, declare two integer variables and call `isGreater` in both scenarios: when `a > b` and when `a <= b`. Print the returned addresses in each case and observe the results.

Task 10: Decoding a Cryptic Message

You have been working on a small side project with a close friend who enjoys leaving coding puzzles. One day, he sends you a mysterious message encoded as four integers:

1869440333 1109424498 1801678700 115

He tells you that the message contains a hidden meaning but gives no further instructions. The integers appear random, but you notice that each byte of an integer may correspond to a single character, which could reveal the original message.

Your task is to uncover the hidden message and display it. Think carefully about how the integers might be structured and how to reconstruct the individual characters.

Hint: Recall casting and how different types can be used to interpret the same memory in multiple ways. You may find the image below helpfull.

Memory layout of the 4-integer array (little-endian, each box = 1 byte)

0x4D	0x65	0x6D	0x6F	0x72	0x79	0x20	0x42	0x6C	0x6F	0x63	0x6B	0x73	0x00	0x00	0x00
0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0x10	0x11	0x12	0x13	0x14	0x15